

LECTURE 7

Relational Algebra: Unary and Set Operations

CSC270 · Database Systems

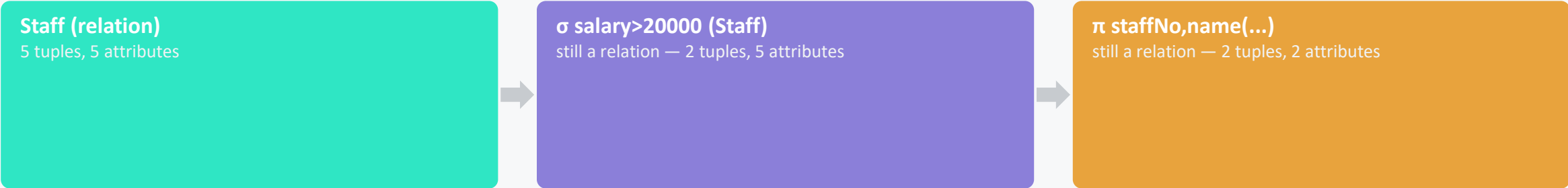
COMSATS University Islamabad, Lahore Campus

Agenda

- What relational algebra is, and its defining closure property
- SELECT (σ) — filtering tuples by a predicate
- PROJECT (π) — filtering attributes (columns)
- Composing SELECT and PROJECT together
- Set operations — UNION, INTERSECTION, SET DIFFERENCE — and union-compatibility
- CARTESIAN PRODUCT ($\times$) — combining every tuple of one relation with every tuple of another

Relational Algebra: Closure

A procedural query language: every operator takes relation(s) and produces a relation. That's the closure property — an operator's output can feed directly into the next operator, just like $(3+4)*2$ in arithmetic.



Family	Operators	Covered in
Unary (one input)	SELECT (σ), PROJECT (π)	This lecture
Set (two union-compatible inputs)	UNION ($\cup$), INTERSECTION ($\cap$), DIFFERENCE ($-$)	This lecture
Product (two inputs, any schema)	CARTESIAN PRODUCT ($\times$)	This lecture
Join / Division (two related inputs)	$\bowtie$, θ -join, $\div$	Lecture 8

SELECT (σ) — Filtering Rows

Extracts the subset of tuples satisfying a predicate — filters rows, keeps every attribute. Same degree in and out, fewer (or equal) tuples.

Query: find every staff member earning more than 20,000.

σ salary>20000 (Staff)

Input: Staff

staffNo	name	position	salary	branchNo
SL21	John White	Manager	30000	B005
SG37	Ann Beech	Assistant	12000	B003
SG14	David Ford	Supervisor	18000	B003
SA9	Mary Howe	Assistant	9000	B007
SL41	Julie Lee	Manager	22000	B005

Output: σ salary>20000 (Staff)

staffNo	name	position	salary	branchNo
SL21	John White	Manager	30000	B005
SL41	Julie Lee	Manager	22000	B005

SELECT here is not the SQL SELECT σ corresponds to SQL's WHERE clause (row filtering). SQL's SELECT keyword corresponds to algebra's PROJECT (π) — choosing columns.

SELECT with a Compound Predicate

Predicates combine =, ≠, <, >, ≤, ≥ with ∧ (AND), ∨ (OR), ¬ (NOT).

Query: staff at branch B003 earning more than 15,000.

```
σ branchNo='B003' ∧ salary>15000 (Staff)
```

Output: exactly one row survives

staffNo	name	position	salary	branchNo
SG14	David Ford	Supervisor	18000	B003

Tracing by hand: SL21 → B005, fails. SG37 → B003 ✓, salary 12000, fails. SG14 → B003 ✓, salary 18000>15000 ✓ — kept. SA9 → B007, fails. SL41 → B005, fails. Result: {SG14} only.

PROJECT (π) — Filtering Columns

Extracts a subset of attributes, discarding the rest — keeps every tuple that passes, narrowed to fewer columns.

Query: list just the name and salary of every staff member.

```
 $\pi$  name, salary (Staff)
```

Output: π name, salary (Staff)

name	salary
John White	30000
Ann Beech	12000
David Ford	18000
Mary Howe	9000
Julie Lee	22000

PROJECT Automatically Removes Duplicates

A relation can never contain duplicate tuples — so PROJECT removes any duplicate rows that result from dropping columns.

Query: which distinct property types does the agency rent?

π type (PropertyForRent)

Input: PropertyForRent (relevant columns)

propertyNo	type
PA14	House
PL94	Flat
PG4	House
PG36	Flat
PG16	House
PG21	Flat

Output: π type (PropertyForRent) — 6 rows collapse to 2

type
House
Flat

House appears 3 times and Flat 3 times in the source — the projected relation, being a true set, keeps each distinct value once. This is why SQL sometimes needs SELECT DISTINCT to be algebra-faithful.

Composing SELECT and PROJECT

Query: staff number and name of every staff member earning more than 20,000.

```
 $\pi$  staffNo, name ( $\sigma$  salary>20000 (Staff))
```

Read inside-out: σ filters first, then π narrows columns

staffNo	name
SL21	John White
SL41	Julie Lee

Order matters for what's still available

π staffNo,name (σ salary>20000 (Staff)) works because σ runs first, while salary is still present. Flip it to σ salary>20000 (π staffNo,name (Staff)) and it fails outright — by the time σ runs, salary has already been projected away.

Set Operations & Union-Compatibility

UNION ($\cup$), INTERSECTION ($\cap$), and SET DIFFERENCE ($-$) are borrowed from set theory — a relation is a set of tuples. All three require the two inputs to be union-compatible.

- 1. Same degree — the same number of attributes.
- 2. Same domain per position — the i -th attribute of each relation holds comparable values (names don't have to match).

Comparison	Union-compatible?
π staffNo (Staff) vs. π clientNo (Client)	Yes — both degree 1, both short alphanumeric-ID domains
Staff (degree 5) vs. Branch (degree 4) <i>Attempting $Staff \cup Branch$ is simply not a legal expression.</i>	No — different degree, and salary vs. postcode domains don't line up

UNION (U)

Query: staff at B003, OR earning more than 20,000 (or both).

```
π staffNo (σ branchNo='B003' (Staff)) U π staffNo (σ salary>20000 (Staff))
```

σ branchNo='B003'

staffNo
SG37
SG14

σ salary>20000

staffNo
SL21
SL41

Output: UNION

staffNo
SG37
SG14
SL21
SL41

No overlap between the two inputs here, so the union is simply all four rows combined: {SG37,SG14} U {SL21,SL41} = {SG37,SG14,SL21,SL41}.

INTERSECTION (∩)

Query: staff at B003 AND earning more than 15,000.

```
π staffNo (σ branchNo='B003' (Staff)) ∩ π staffNo (σ salary>15000 (Staff))
```

{SG37, SG14}

staffNo
SG37
SG14

{SL21, SG14, SL41}

staffNo
SL21
SG14
SL41

Output: INTERSECTION

staffNo
SG14

Only SG14 appears in both sets — SG37 is B003 but doesn't clear 15,000; SL21/SL41 clear 15,000 but aren't at B003.

SET DIFFERENCE (–)

Query: staff at B003 but NOT earning more than 15,000. Order matters — $R - S \neq S - R$ in general.

```
π staffNo (σ branchNo='B003' (Staff)) – π staffNo (σ salary>15000 (Staff))
```

{SG37, SG14}

staffNo
SG37
SG14

{SL21, SG14, SL41}

staffNo
SL21
SG14
SL41

Output: DIFFERENCE

staffNo
SG37

Sanity-check identity $(R \cap S)$ and $(R - S)$ together always reconstruct R exactly, no overlap: $\{SG14\} + \{SG37\} = \{SG37, SG14\}$ — the original B003 staff set. Use this to double-check by-hand tracing.

CARTESIAN PRODUCT (×)

Combines EVERY tuple of R with EVERY tuple of S. Degree = sum of degrees; cardinality = product of cardinalities. No union-compatibility required.

```
π clientNo (Client) × π propertyNo (σ city='Lahore' (PropertyForRent))
```

clientNo (3)

clientNo
CR56
CR62
CR76



Lahore propertyNo (3)

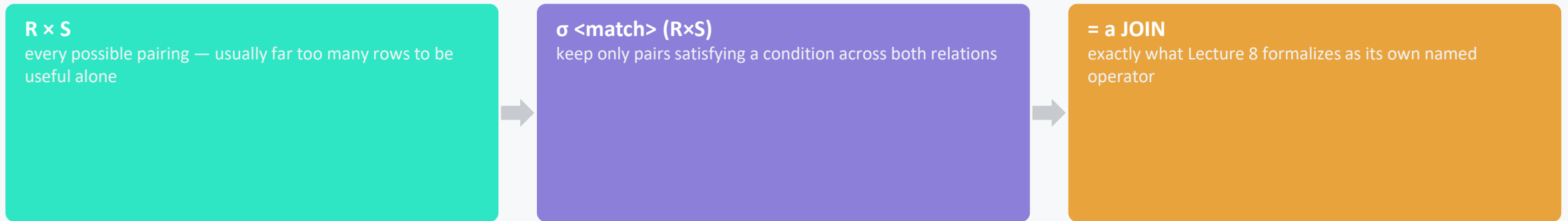
propertyNo
PA14
PG4
PG36

Output: 3 × 3 = 9 tuples

clientNo	propertyNo
CR56	PA14
CR56	PG4
CR56	PG36
CR62	PA14
CR62	PG4
CR62	PG36
CR76	PA14
CR76	PG4
CR76	PG36

Every 3×3=9 pairing is present, no filtering — including nonsense pairs like CR76 (maxRent 20,000) with PG4 (rent 35,000). This is exactly the limitation Lecture 8's joins fix.

Where Lecture 8 Picks Up



KEY TAKEAWAYS

Relational Algebra: Unary & Set Operations

- Relational algebra is procedural: every operator takes relation(s) and produces a relation — the closure property that lets operators nest arbitrarily deeply.
- SELECT (σ) filters rows by a predicate; PROJECT (π) filters columns and auto-removes any resulting duplicate tuples.
- SQL's SELECT/WHERE split maps onto algebra's π/σ almost exactly backwards from the naming.
- UNION, INTERSECTION, and SET DIFFERENCE require union-compatible inputs: same degree, matching domains.
- CARTESIAN PRODUCT needs no compatibility — it pairs every tuple with every tuple, and is almost always followed by a SELECT, which is exactly the idea Lecture 8's joins formalize.